

CHƯƠNG 1. GIỚI THIỆU VỀ HỆ THỐNG	1
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	2
I. Xác thực và phân quyền	2
1. Đăng nhập	2
2. Đăng ký tài khoản	5
3. Profile	9
II. Giám sát cảm biến	13
1. Xem dữ liệu cảm biến	13
2. Xem biểu đồ	17
3. Xem lịch sử cảm biến	20
III. Điều khiển thiết bị	23
1. Bật tắt thiết bị	23
2. Xem lịch sử điều khiển thiết bị	28
CHƯƠNG 3. THIẾT KẾ CƠ SỞ DỮ LIỆU	31
I. Phân tích lớp thực thể	31
1. Phân tích các lớp thực thể	31
2. Phân tích mối quan hệ giữa các lớp thực thể	37
II. Sơ đồ lớp thực thể	38
III. Sơ đồ ERD	39
CHƯƠNG 4. THIẾT KẾ API	40
1. API Đăng nhập	40
2. Profile	43
3. Xem dữ liệu cảm biến	46
4. Xem biểu đồ	48
5. Xem lịch sử dữ liệu cảm biến	52
6. Điều khiển thiết bị	55
7. Xem lịch sử điều khiển thiết bị	58

CHƯƠNG 1. GIỚI THIỆU VỀ HỆ THỐNG

Trong bối cảnh Internet of Things (IoT) ngày càng được ứng dụng rộng rãi, việc kết nối các thiết bị vật lý với hệ thống phần mềm giúp con người có thể thu thập, giám sát và điều khiển thiết bị một cách thuận tiện. Xuất phát từ nhu cầu đó, nhóm xây dựng hệ thống IoT giám sát môi trường và điều khiển thiết bị từ xa.

Hệ thống sử dụng ESP8266/ESP32 kết hợp với các cảm biến để thu thập các thông số môi trường như nhiệt độ, độ ẩm và ánh sáng. Dữ liệu được thiết bị đọc trực tiếp từ cảm biến và truyền về hệ thống thông qua giao thức MQTT. Backend tiếp nhận, xử lý và lưu trữ dữ liệu vào cơ sở dữ liệu, đồng thời cung cấp các RESTful API để Frontend truy xuất và hiển thị thông tin.

Thông qua giao diện Web, người dùng có thể theo dõi dữ liệu cảm biến theo thời gian thực, xem biểu đồ biến động, tra cứu lịch sử dữ liệu và điều khiển thiết bị như bật/tắt đèn LED. Các thao tác điều khiển cũng được ghi nhận và lưu trữ, giúp người dùng có thể theo dõi lịch sử hoạt động của thiết bị.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Hệ thống được phân thành 3 loại nhóm chức năng chính: Xác thực, Giám sát cảm biến, Quản lý dữ liệu và Điều khiển thiết bị

I. Xác thực và phân quyền

1. Đăng nhập

a) Mô tả tính năng

Cho phép người dùng đăng nhập vào hệ thống bằng username và password để xác thực danh tính và truy cập các chức năng được cấp quyền.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng truy cập trang Login của hệ thống.
2. Người dùng nhập username/email và password.
3. Người dùng nhấn nút Login.
4. Frontend kiểm tra dữ liệu đầu vào và gửi yêu cầu đăng nhập đến Backend thông qua RESTful API.
5. Backend tiếp nhận yêu cầu và tìm kiếm tài khoản người dùng trong Database.

6. Database thực hiện truy vấn thông tin tài khoản dựa trên username/email.
7. Database trả thông tin tài khoản về Backend.
8. Backend kiểm tra mật khẩu người dùng cung cấp với mật khẩu đã được lưu trong hệ thống.
9. Nếu thông tin đăng nhập hợp lệ, Backend tạo JWT/Access Token cho người dùng.
10. Backend trả kết quả đăng nhập thành công và token về Frontend.
11. Frontend lưu token và chuyển người dùng đến DashboardPage.
12. DashboardPage hiển thị giao diện và các chức năng mà người dùng được phép sử dụng.

Trường hợp đăng nhập thất bại:

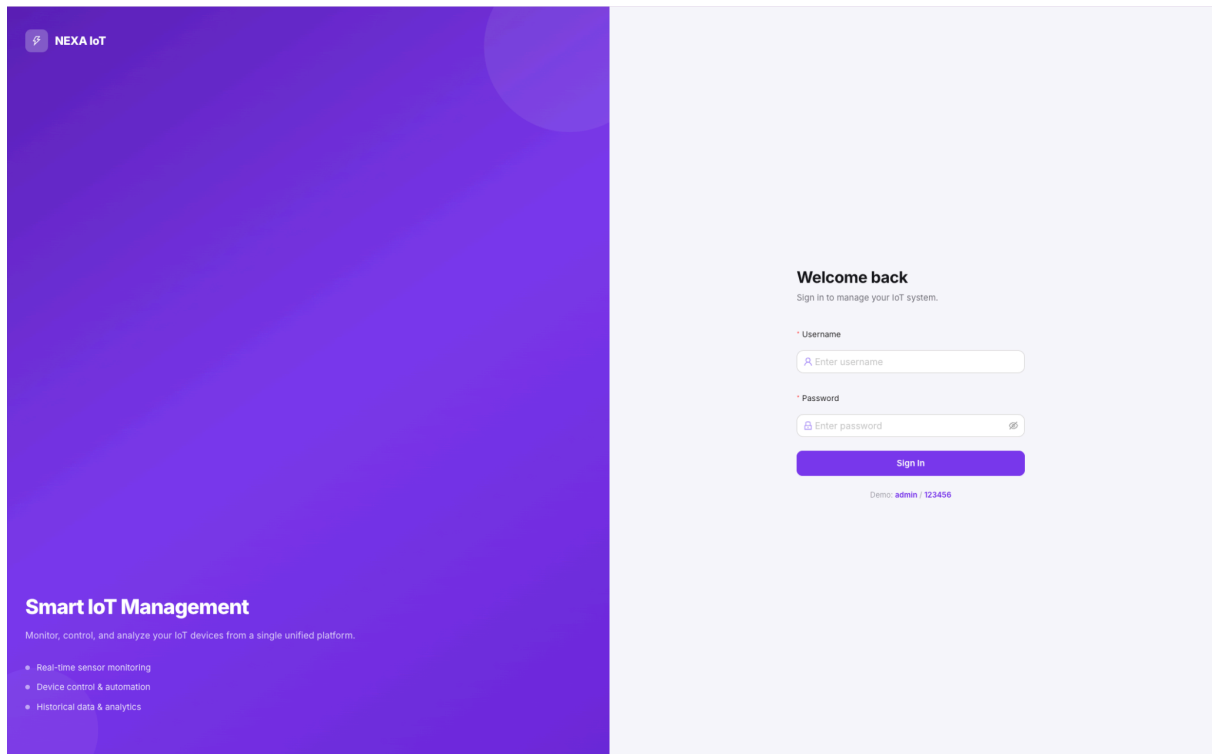
- Nếu tài khoản không tồn tại hoặc mật khẩu không chính xác, Backend trả về lỗi xác thực.
- Frontend hiển thị thông báo “Username hoặc password không chính xác” và yêu cầu người dùng nhập lại.

Các thông tin được sử dụng gồm:

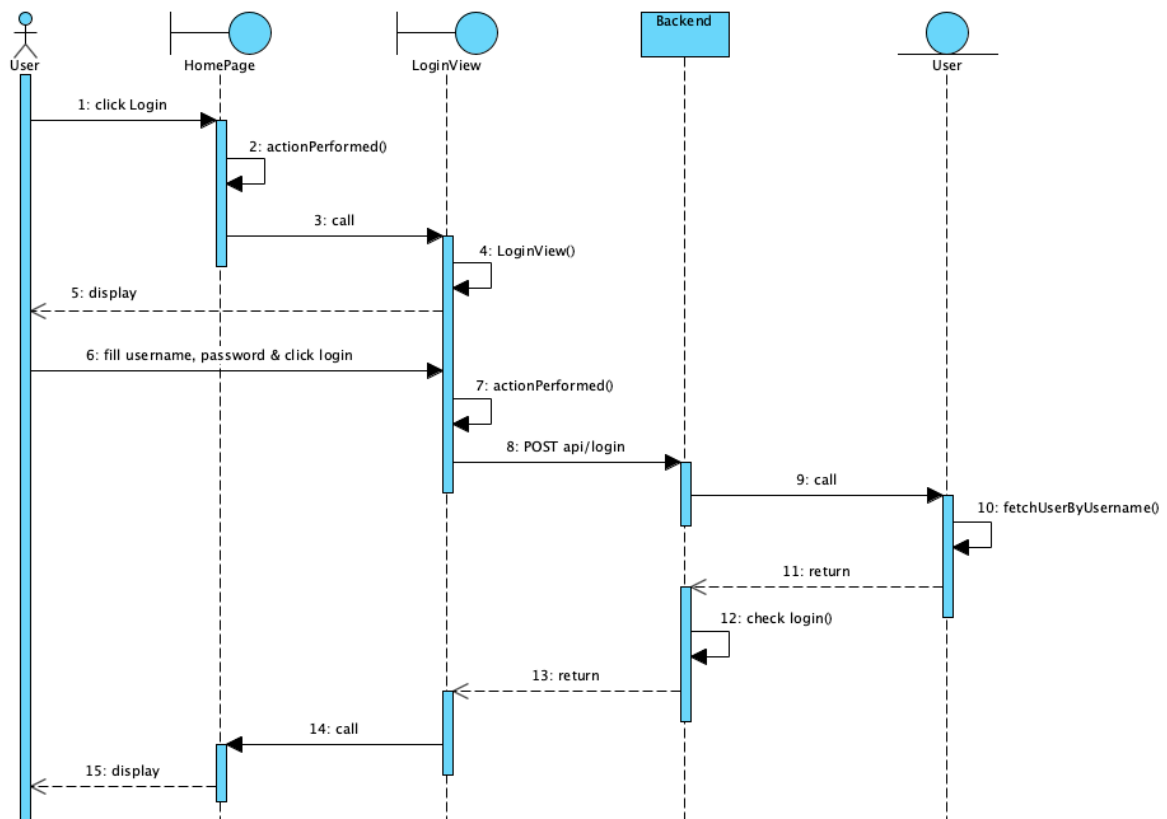
- Username/Email: Tài khoản dùng để đăng nhập.
- Password: Mật khẩu của tài khoản.
- Access Token/JWT: Token được Backend cấp sau khi đăng nhập thành công.
- Kết quả đăng nhập: Thành công hoặc thất bại.
- Thông báo: Thông báo kết quả đăng nhập cho người dùng.

API đề xuất: POST api/auth/login

c) Giao diện người dùng



d) Sequence Diagram



Mô tả tuần tự:

1. Người dùng thao tác chọn chức năng đăng nhập trên giao diện trang chủ.
2. Hệ thống ghi nhận sự kiện tương tác vừa diễn ra trên trang chủ.
3. Trang chủ thực hiện lệnh gọi để chuyển hướng sang màn hình đăng nhập.
4. Màn hình giao diện đăng nhập bắt đầu được hệ thống khởi tạo.
5. Giao diện đăng nhập xuất hiện và hiển thị trên màn hình để người dùng tương tác.
6. Người dùng tiến hành điền thông tin tài khoản (bao gồm tên đăng nhập, mật khẩu) và nhấn nút xác nhận đăng nhập.
7. Giao diện đăng nhập ghi nhận sự kiện và thu thập dữ liệu do người dùng vừa nhập.
8. Giao diện đóng gói thông tin và gửi yêu cầu xác thực tài khoản đến máy chủ (Backend).
9. Máy chủ tiếp nhận yêu cầu và gửi lệnh truy vấn tới bộ phận quản lý dữ liệu (Thực thể User / Cơ sở dữ liệu).
10. Cơ sở dữ liệu thực thi quá trình tìm kiếm thông tin tài khoản dựa trên tên đăng nhập vừa được cung cấp.
11. Cơ sở dữ liệu trả về thông tin của người dùng tương ứng cho máy chủ.
12. Máy chủ tiến hành kiểm tra, đối chiếu dữ liệu (như kiểm tra mật khẩu) để xác thực tính hợp lệ của phiên đăng nhập.
13. Máy chủ hoàn tất quá trình kiểm tra và trả kết quả phản hồi về cho giao diện đăng nhập.
14. Khi nhận được kết quả hợp lệ, giao diện đăng nhập thực hiện lệnh điều hướng người dùng trở lại giao diện trang chủ.
15. Giao diện trang chủ cập nhật trạng thái và hiển thị lại lên màn hình cho người dùng.

2. Đăng ký tài khoản

a) Mô tả tính năng

Cho phép người dùng tạo tài khoản mới trên hệ thống bằng cách cung cấp các thông tin đăng nhập (username, password) để hệ thống lưu trữ, từ đó người dùng có thể sử dụng để đăng nhập và truy cập các chức năng của ứng dụng.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng đang ở trang chủ (HomePage) và nhấn chọn chức năng Đăng ký tài khoản.
2. Hệ thống xử lý sự kiện và hiển thị màn hình giao diện Đăng ký (RegisterView) cho người dùng.
3. Người dùng nhập các thông tin bao gồm: username, password, confirmPassword và nhấn nút xác nhận (OK).
4. Frontend (RegisterView) thực hiện kiểm tra tính hợp lệ của dữ liệu đầu vào (validate).
5. Nếu dữ liệu hợp lệ, Frontend gửi yêu cầu tạo tài khoản mới đến Backend thông qua RESTful API (phương thức POST api/register).
6. Backend tiếp nhận yêu cầu và gọi xuống Database (Thực thể User) để kiểm tra xem tên đăng nhập này đã tồn tại trong hệ thống hay chưa (checkExistsByUsername()).
7. Database thực hiện truy vấn và trả kết quả kiểm tra về cho Backend.
8. Nếu username chưa tồn tại, Backend thực hiện lệnh lưu thông tin tài khoản người dùng mới vào Database (save()).
9. Database lưu trữ dữ liệu thành công và trả phản hồi về Backend.
10. Backend trả kết quả đăng ký thành công về cho Frontend (RegisterView).
11. Frontend hiển thị thông báo "Đăng ký thành công" cho người dùng.
12. Người dùng nhấn nút "OK" để xác nhận đóng thông báo.
13. Frontend tự động chuyển hướng người dùng quay trở lại trang chủ (HomePage).
14. HomePage hiển thị lại giao diện ban đầu.

Trường hợp đăng ký thất bại:

- Nếu dữ liệu đầu vào không hợp lệ (ví dụ: password và confirmPassword không khớp, hoặc thông tin bị bỏ trống), Frontend (ở bước validate) sẽ chặn lại, hiển thị thông báo lỗi và yêu cầu người dùng nhập lại.
- Nếu username mà người dùng nhập đã tồn tại trong hệ thống, Backend sẽ trả về lỗi trùng lặp dữ liệu. Frontend nhận lỗi, hiển thị thông báo “Tên đăng nhập đã tồn tại” và yêu cầu người dùng chọn tên khác.
- Nếu có lỗi trong quá trình lưu dữ liệu xuống Database, Backend trả về lỗi hệ thống và Frontend hiển thị thông báo thử lại sau.

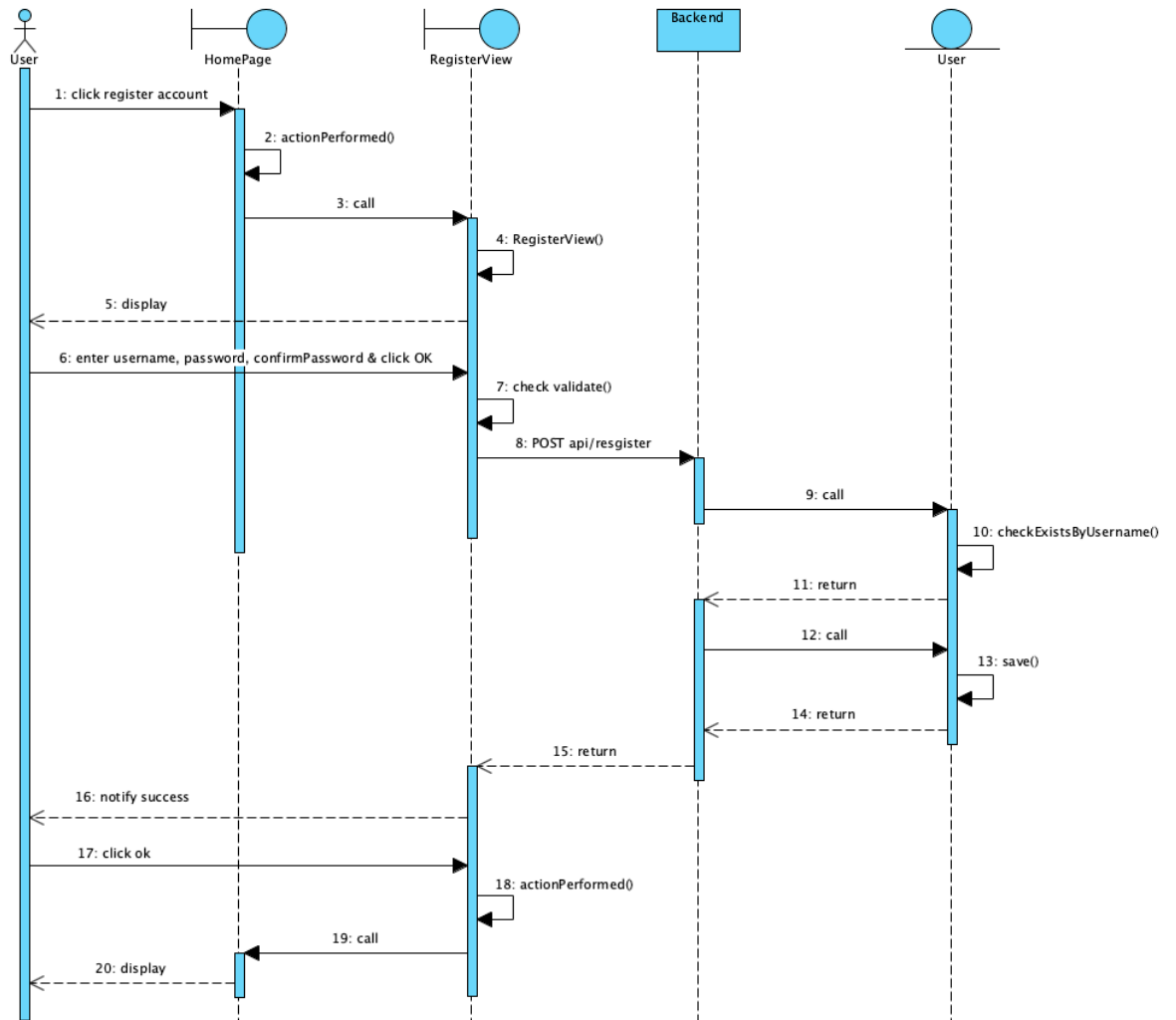
Các thông tin được sử dụng gồm:

- Username: Tên đăng nhập người dùng muốn tạo.
- Password: Mật khẩu bảo mật cho tài khoản.
- Confirm Password: Chuỗi nhập lại mật khẩu để xác minh tính chính xác.
- Kết quả đăng ký: Trạng thái hệ thống trả về (Thành công hoặc thất bại).
- Thông báo: Nội dung hiển thị kết quả hoặc lỗi gửi đến người dùng trong quá trình thao tác.

API đề xuất: POST api/auth/register

c) Giao diện người dùng

d) Sequence Diagram



Mô tả tuần tự:

1. Người dùng thao tác chọn chức năng đăng ký tài khoản trên giao diện trang chủ.
2. Hệ thống ghi nhận sự kiện tương tác vừa diễn ra trên trang chủ.
3. Trang chủ thực hiện lệnh gọi để chuyển hướng sang màn hình đăng ký.
4. Màn hình giao diện đăng ký bắt đầu được hệ thống khởi tạo.
5. Giao diện đăng ký xuất hiện và hiển thị trên màn hình để người dùng tương tác.
6. Người dùng tiến hành điền các thông tin cần thiết bao gồm: tên đăng nhập, mật khẩu, xác nhận mật khẩu và nhấn nút xác nhận (OK).
7. Giao diện đăng ký tự động kiểm tra tính hợp lệ của các dữ liệu mà người dùng vừa nhập (ví dụ: mật khẩu có khớp nhau không, độ dài có đạt yêu cầu không).

8. Sau khi kiểm tra dữ liệu hợp lệ ở phía giao diện, hệ thống đóng gói thông tin và gửi yêu cầu đăng ký tài khoản mới đến máy chủ (Backend).
9. Máy chủ tiếp nhận yêu cầu và gửi lệnh truy vấn tới cơ sở dữ liệu (Thực thể User).
10. Cơ sở dữ liệu tiến hành kiểm tra xem tên đăng nhập mà người dùng vừa nhập đã tồn tại trong hệ thống hay chưa.
11. Cơ sở dữ liệu trả kết quả kiểm tra (tài khoản chưa tồn tại/hợp lệ) về cho máy chủ.
12. Máy chủ tiếp tục gửi lệnh yêu cầu lưu trữ thông tin người dùng mới xuống cơ sở dữ liệu.
13. Cơ sở dữ liệu thực thi quá trình lưu thông tin tài khoản mới vào hệ thống.
14. Cơ sở dữ liệu lưu thành công và trả phản hồi về cho máy chủ.
15. Máy chủ hoàn tất quá trình xử lý và trả kết quả phản hồi đăng ký thành công về cho giao diện đăng ký.
16. Giao diện đăng ký hiển thị thông báo "Đăng ký thành công" trên màn hình cho người dùng.
17. Người dùng tương tác nhấn "OK" để xác nhận và đóng thông báo.
18. Giao diện đăng ký ghi nhận hành động đóng thông báo của người dùng.
19. Giao diện đăng ký thực hiện lệnh điều hướng, đưa người dùng quay trở lại giao diện trang chủ.
20. Giao diện trang chủ được cập nhật và hiển thị lại lên màn hình.

3. Profile

a) Mô tả tính năng

Cho phép người dùng xem và quản lý thông tin cá nhân của tài khoản, bao gồm họ tên, username, email, số điện thoại, ảnh đại diện, GitHub, Figma và mật khẩu. Người dùng có thể cập nhật các thông tin cá nhân và thay đổi mật khẩu nhằm đảm bảo thông tin tài khoản luôn chính xác, đầy đủ và an toàn.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng chọn chức năng Profile trên giao diện hệ thống.
2. Frontend gửi yêu cầu lấy thông tin cá nhân đến Backend thông qua RESTful API.
3. Backend tiếp nhận yêu cầu và xác thực người dùng thông qua JWT/Access Token.
4. Backend lấy thông tin userId từ token và truy vấn thông tin tài khoản trong Database.
5. Database thực hiện truy vấn tìm kiếm thông tin người dùng theo userId.
6. Database trả thông tin tài khoản về Backend.
7. Backend xử lý dữ liệu và trả thông tin Profile về Frontend.
8. Frontend hiển thị thông tin cá nhân của người dùng trên trang Profile.
9. Người dùng có thể xem các thông tin như họ tên, username, email, số điện thoại, ảnh đại diện, GitHub và Figma.
10. Khi muốn thay đổi thông tin, người dùng chọn Edit Profile.
11. Người dùng chỉnh sửa các thông tin cá nhân và nhấn Save.
12. Frontend gửi yêu cầu cập nhật thông tin đến Backend thông qua RESTful API.
13. Backend xác thực người dùng và kiểm tra tính hợp lệ của dữ liệu được gửi lên.
14. Backend yêu cầu Database cập nhật thông tin cá nhân.
15. Database thực hiện cập nhật và trả kết quả về Backend.
16. Backend trả kết quả cập nhật về Frontend.
17. Frontend cập nhật lại thông tin Profile và hiển thị thông báo cập nhật thành công.
18. Khi muốn thay đổi mật khẩu, người dùng chọn chức năng Change Password.
19. Người dùng nhập mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới.
20. Frontend gửi yêu cầu đổi mật khẩu đến Backend.
21. Backend xác thực người dùng và kiểm tra mật khẩu hiện tại.

22. Backend kiểm tra mật khẩu mới và mật khẩu xác nhận có khớp hay không.
23. Nếu hợp lệ, Backend mã hóa mật khẩu mới trước khi lưu vào Database.
24. Database cập nhật mật khẩu mới và trả kết quả về Backend.
25. Backend trả kết quả đổi mật khẩu về Frontend.
26. Frontend hiển thị thông báo đổi mật khẩu thành công cho người dùng.

Các thông tin được hiển thị gồm:

- **Ảnh đại diện:** Ảnh đại diện của người dùng.
- **Họ tên:** Họ và tên của người dùng.
- **Username:** Tên tài khoản đăng nhập.
- **Email:** Địa chỉ email của người dùng.
- **Số điện thoại:** Số điện thoại liên hệ.
- **GitHub:** Đường dẫn đến tài khoản GitHub.
- **Figma:** Đường dẫn đến tài khoản hoặc trang Figma.
- **Mật khẩu:** Được ẩn và không hiển thị trực tiếp vì lý do bảo mật.

Các thao tác:

- **View Profile:** Xem thông tin cá nhân.
- **Edit Profile:** Chỉnh sửa thông tin cá nhân.
- **Change Avatar:** Thay đổi ảnh đại diện.
- **Change Password:** Thay đổi mật khẩu.
- **Save:** Lưu các thay đổi.
- **Cancel:** Hủy thay đổi.

Kết quả:

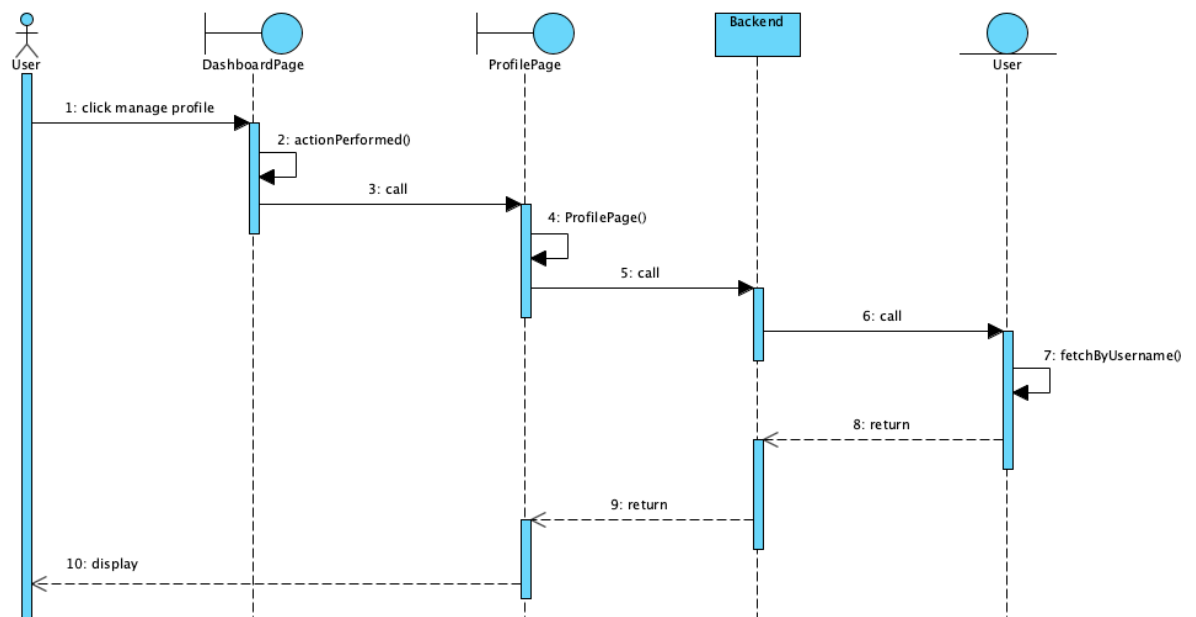
- **Thành công:** Thông tin được cập nhật và giao diện hiển thị dữ liệu mới.
- **Thất bại:** Backend trả về lỗi và Frontend hiển thị thông báo tương ứng, ví dụ email đã tồn tại, dữ liệu không hợp lệ hoặc mật khẩu hiện tại không chính xác.

API đề xuất: GET api/auth/profile

c) Giao diện người dùng

The screenshot shows a web application interface for managing a user profile. On the left is a sidebar with a main menu containing 'Dashboard', 'Sensors', 'Control History', 'Profile' (selected), 'Settings', and 'Logout'. The main content area is titled 'Profile' with the subtitle 'Manage your account and preferences.' It features a user card for 'Administrator' (@admin, Super Admin) with contact details. Below the card are two form sections: 'Edit Profile' and 'Change Password'. The 'Edit Profile' form includes fields for Full Name, Username, Email, Phone, GitHub, and Figma, with a 'Save Changes' button. The 'Change Password' form includes fields for Current Password, New Password, and Confirm Password, with a 'Change Password' button. The top right corner shows a user profile icon for 'Admin Super Admin'.

d) Sequence Diagram



Mô tả tuần tự:

1. Người dùng đang ở giao diện trang tổng quan và nhấn chọn chức năng quản lý hồ sơ cá nhân (manage profile).

2. Giao diện trang tổng quan ghi nhận sự kiện click của người dùng.
3. Trang tổng quan thực hiện lệnh gọi để chuyển hướng người dùng sang màn hình giao diện Hồ sơ cá nhân (ProfilePage).
4. Giao diện Hồ sơ cá nhân bắt đầu được hệ thống khởi tạo.
5. Ngay khi khởi tạo, giao diện này tự động gửi một yêu cầu (API) đến máy chủ (Backend) để lấy dữ liệu thông tin người dùng.
6. Máy chủ tiếp nhận yêu cầu và gọi xuống Cơ sở dữ liệu (Thực thể User) để yêu cầu trích xuất thông tin.
7. Cơ sở dữ liệu thực thi truy vấn tìm kiếm hồ sơ người dùng dựa trên tên đăng nhập (username) của phiên hoạt động hiện tại.
8. Cơ sở dữ liệu tìm thấy dữ liệu và trả thông tin hồ sơ về cho máy chủ.
9. Máy chủ nhận dữ liệu, xử lý và trả kết quả phản hồi về cho giao diện Hồ sơ cá nhân.
10. Giao diện nhận được dữ liệu và hiển thị đầy đủ thông tin cá nhân lên màn hình cho người dùng theo dõi.

II. Giám sát cảm biến

Chức năng giám sát dữ liệu cảm biến cho phép người dùng theo dõi các thông số môi trường được thu thập trực tiếp từ các cảm biến thông qua ESP8266/ESP32. Hệ thống tập trung giám sát ba thông số chính gồm nhiệt độ, độ ẩm và cường độ ánh sáng.

ESP8266/ESP32 định kỳ đọc dữ liệu từ các cảm biến và gửi dữ liệu lên hệ thống thông qua giao thức MQTT. Backend tiếp nhận dữ liệu, xử lý và lưu trữ vào cơ sở dữ liệu. Frontend sử dụng RESTful API để lấy dữ liệu và hiển thị cho người dùng trên Dashboard.

1. Xem dữ liệu cảm biến

a) Mô tả tính năng

Chức năng Xem dữ liệu cảm biến cho phép người dùng theo dõi các thông số môi trường được thu thập từ hệ thống IoT, bao gồm nhiệt độ, độ ẩm và cường độ ánh sáng.

Dữ liệu được thu thập từ các cảm biến thông qua ESP8266/ESP32 và truyền về hệ thống bằng giao thức MQTT. Người dùng có thể xem các giá trị cảm biến mới nhất trên giao diện Dashboard.

b) Mô tả chi tiết

Luồng xử lý:

1. Cảm biến thực hiện đo các thông số môi trường.
2. ESP8266/ESP32 đọc giá trị từ cảm biến.
3. ESP8266/ESP32 đóng gói dữ liệu và gửi lên MQTT Broker.
4. Backend nhận dữ liệu từ MQTT Broker.
5. Backend xử lý và lưu dữ liệu vào Database.
6. Frontend gửi request đến Backend thông qua RESTful API để lấy dữ liệu mới nhất.
7. Backend truy vấn Database và trả kết quả về Frontend.
8. Frontend hiển thị dữ liệu trên Dashboard.

Các thông tin được hiển thị gồm:

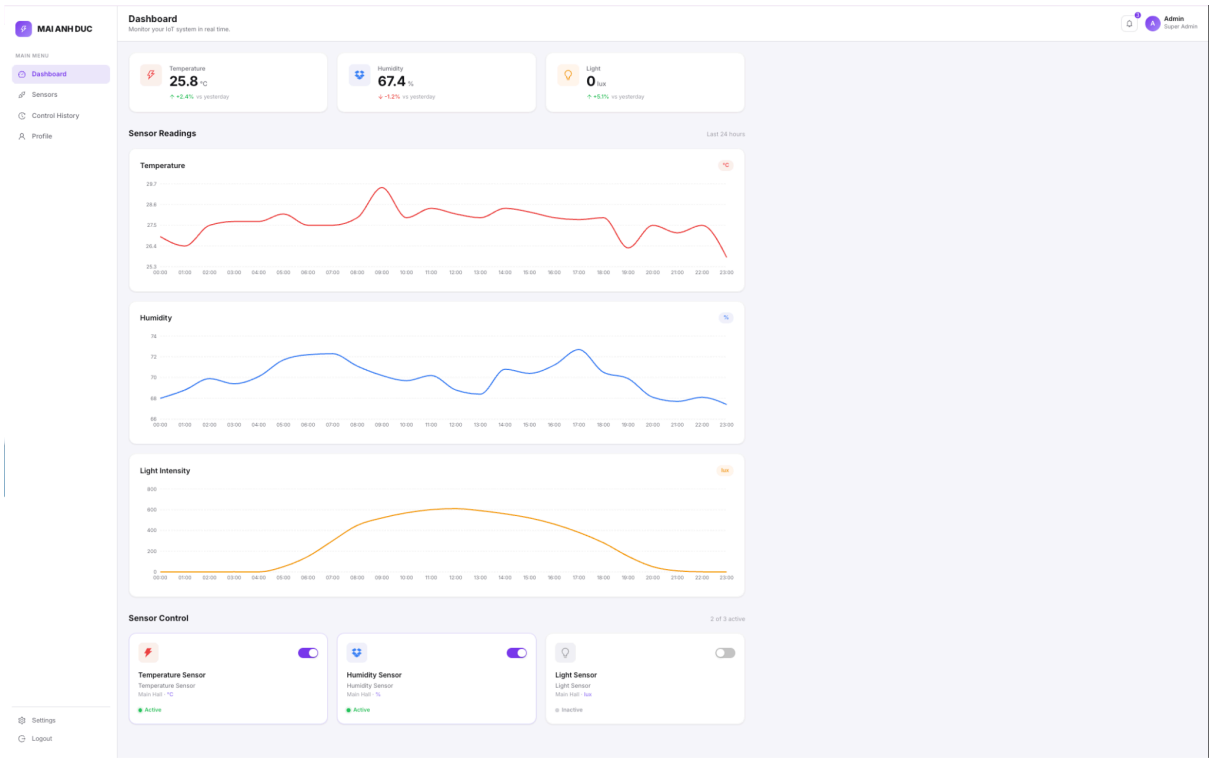
- Tên loại cảm biến.
- Giá trị đo được.
- Đơn vị đo.
- Thời gian đo/cập nhật gần nhất.
- Trạng thái kết nối hoặc trạng thái dữ liệu nếu cần.

Cảm biến	Giá trị	Đơn vị	Thời gian
Nhiệt độ	28	°C	14:20:05

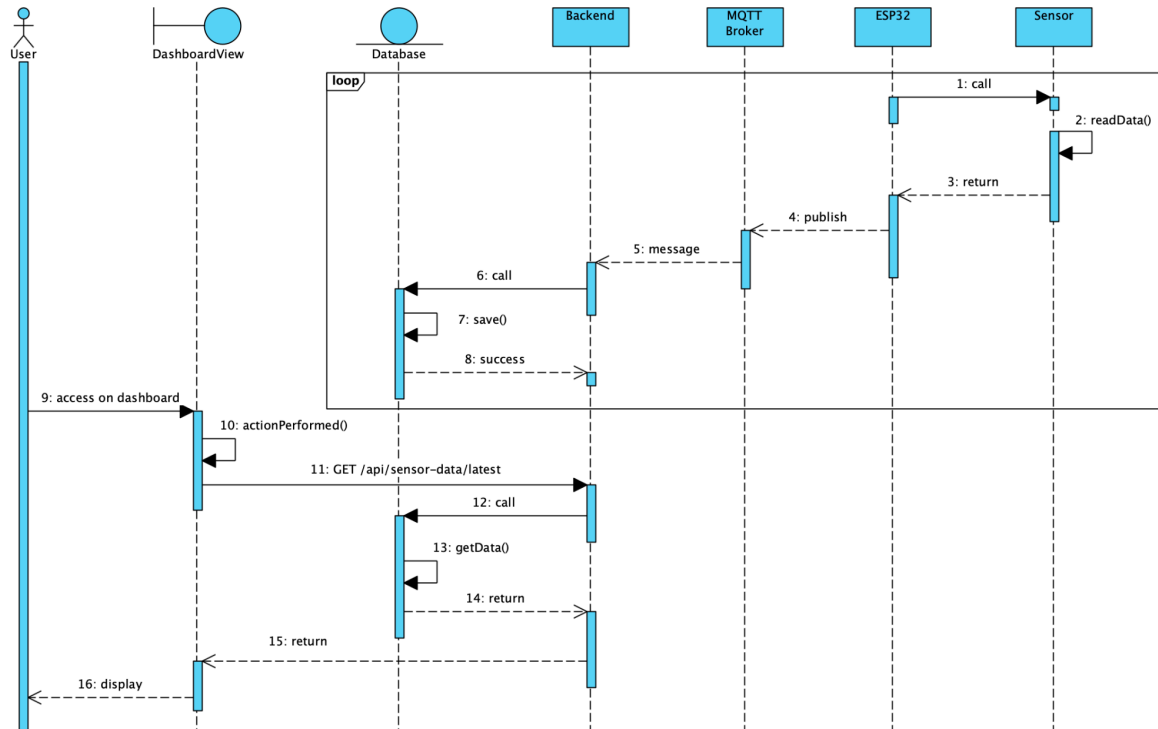
Độ ẩm	72	%	14:20:05
Ánh sáng	430	lux	14:20:05

API đề xuất: GET api/sensor-data/latest

c) Giao diện người dùng



d) Sequence Diagram



Mô tả tuần tự scenario:

1. ESP32 gọi Sensor để yêu cầu Sensor đọc dữ liệu.
2. Sensor thực hiện readData() để đọc giá trị cảm biến.
3. Sensor trả dữ liệu về ESP32 sau khi đọc xong.
4. ESP32 publish dữ liệu lên MQTT Broker thông qua một MQTT topic.
5. MQTT Broker gửi message đến Backend vì Backend đã subscribe topic tương ứng.
6. Backend gọi Database để lưu dữ liệu cảm biến nhận được.
7. Database thực hiện save() và lưu dữ liệu vào hệ thống.
8. Database trả về success cho Backend để xác nhận dữ liệu đã được lưu thành công.
9. User truy cập Dashboard để xem dữ liệu cảm biến.
10. DashboardView thực hiện actionPerformed() để xử lý thao tác của User.
11. DashboardView gửi request GET /api/sensor-data/latest đến Backend để lấy dữ liệu cảm biến mới nhất.
12. Backend gọi Database để truy vấn dữ liệu.
13. Database thực hiện getData() để lấy dữ liệu cảm biến mới nhất.

14. Database trả dữ liệu về Backend sau khi truy vấn thành công.
15. Backend trả dữ liệu về DashboardView thông qua API.
16. DashboardView hiển thị dữ liệu lên Dashboard cho User.

2. Xem biểu đồ

a) Mô tả tính năng

Chức năng Xem biểu đồ cho phép người dùng theo dõi sự thay đổi của các thông số cảm biến theo thời gian thông qua biểu đồ trực quan trên Dashboard.

Người dùng có thể xem dữ liệu của nhiệt độ, độ ẩm và ánh sáng trong một khoảng thời gian nhất định, từ đó dễ dàng nhận biết xu hướng tăng, giảm hoặc biến động của môi trường.

b) Mô tả chi tiết

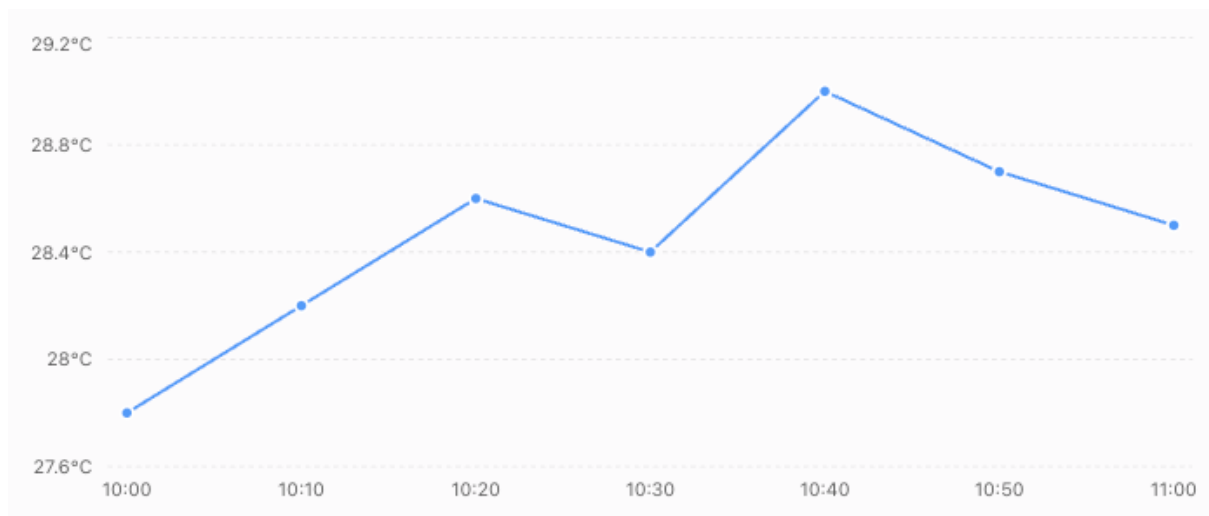
Chức năng cho phép người dùng theo dõi sự thay đổi của dữ liệu cảm biến theo thời gian thông qua biểu đồ. Người dùng có thể lựa chọn loại cảm biến và khoảng thời gian muốn xem.

Luồng xử lý:

1. Người dùng truy cập vào Dashboard.
2. Người dùng lựa chọn loại cảm biến cần xem biểu đồ.
3. Người dùng lựa chọn khoảng thời gian cần xem dữ liệu.
4. Frontend gửi request đến Backend thông qua RESTful API với các điều kiện đã lựa chọn.
5. Backend tiếp nhận request và truy vấn dữ liệu cảm biến trong Database.
6. Database trả về danh sách dữ liệu cảm biến phù hợp với điều kiện tìm kiếm.
7. Backend xử lý dữ liệu và trả kết quả về Frontend.
8. Frontend nhận dữ liệu và biểu diễn dưới dạng biểu đồ.
9. Người dùng theo dõi sự thay đổi của giá trị cảm biến theo thời gian.

Các thông tin được hiển thị gồm:

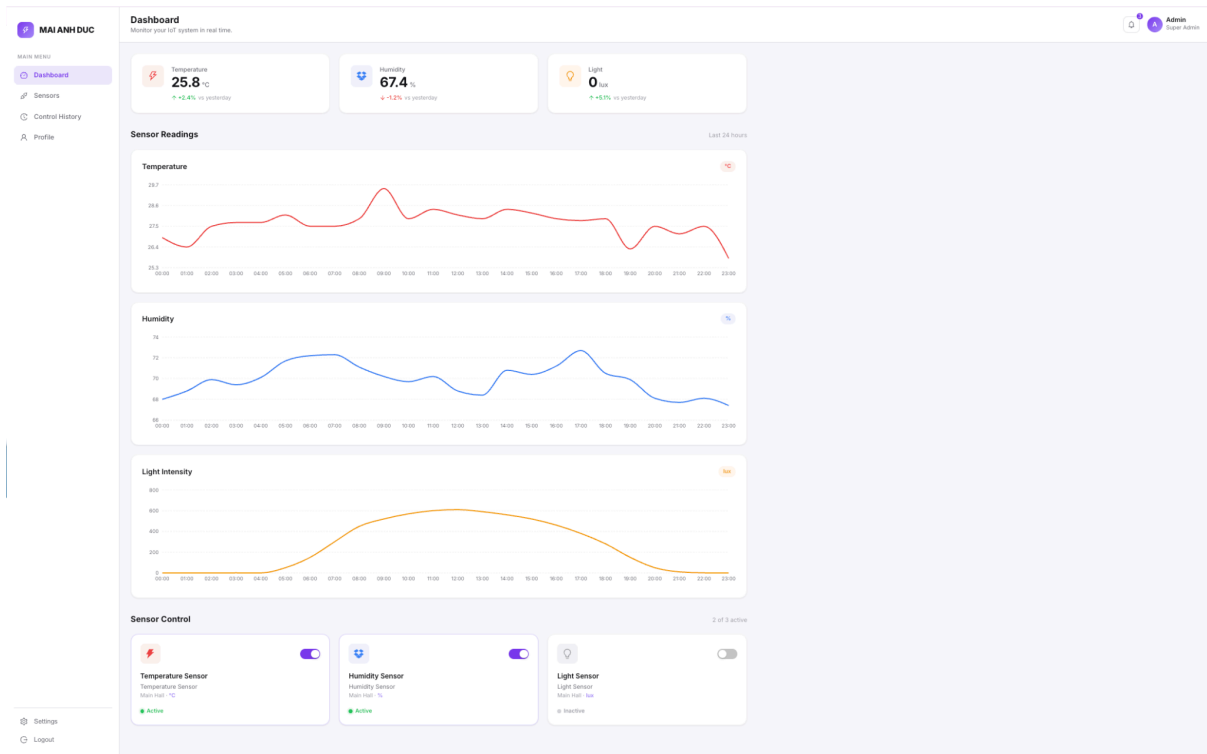
- Loại cảm biến.
- Giá trị cảm biến.
- Thời gian đo.
- Khoảng thời gian được lựa chọn.
- Biểu đồ thể hiện sự thay đổi của giá trị theo thời gian.



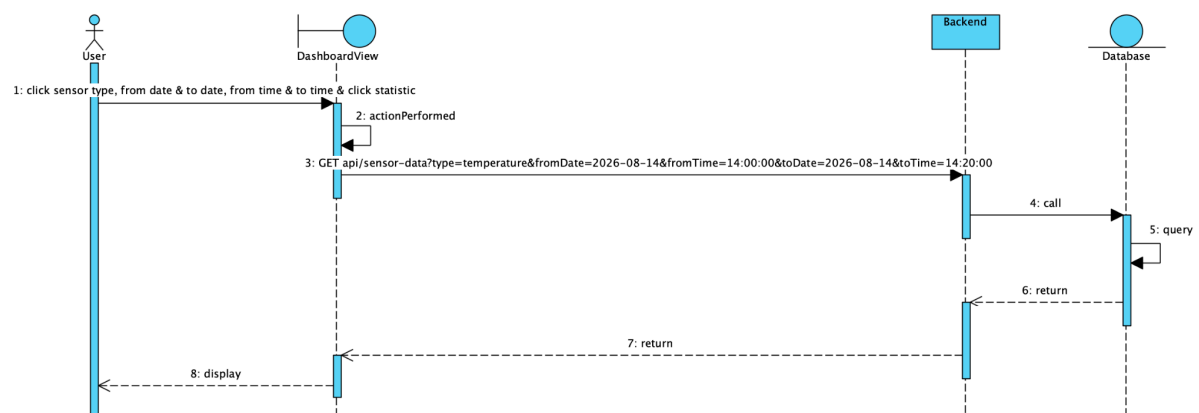
API đề xuất: GET

**api/sensor-data?type=temperature&fromDate=2026-08-14&fromTime=10:00:00
&toDate=2026-08-14&toTime=14:20:00**

c) Giao diện người dùng



d) Sequence diagram



Mô tả tuần tự

1. User chọn loại Sensor, ngày bắt đầu, giờ bắt đầu, ngày kết thúc và giờ kết thúc, sau đó nhấn Statistic.
2. DashboardView thực hiện actionPerformed() để xử lý thao tác của User.
3. DashboardView gửi request đến Backend:

GET

/api/sensor-data?type=temperature&fromDate=2026-08-14&fromTime=14:00:00&toDate=2026-08-14&toTime=14:20:00

→ Yêu cầu Backend lấy dữ liệu nhiệt độ từ 14:00 đến 14:20 ngày 14/08/2026.

4. Backend gọi Database để yêu cầu truy vấn dữ liệu.
5. Database thực hiện query để tìm các bản ghi phù hợp với loại sensor và khoảng thời gian được yêu cầu.
6. Database trả kết quả về Backend.
7. Backend trả dữ liệu về DashboardView thông qua API.
8. DashboardView hiển thị kết quả cho User, ví dụ dưới dạng bảng, biểu đồ hoặc thống kê.

3. Xem lịch sử cảm biến

a) Mô tả tính năng

Chức năng Xem lịch sử cảm biến cho phép người dùng xem lại các dữ liệu cảm biến đã được hệ thống thu thập và lưu trữ trong Database.

Người dùng có thể tìm kiếm dữ liệu theo loại cảm biến và khoảng thời gian, từ đó theo dõi lại các giá trị đã đo trong quá khứ.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng truy cập vào giao diện Data Sensor.
2. Người dùng lựa chọn loại cảm biến cần xem.
3. Người dùng lựa chọn ngày và thời gian bắt đầu.
4. Người dùng lựa chọn ngày và thời gian kết thúc.
5. Người dùng nhấn nút Tìm kiếm.
6. Frontend gửi request đến Backend thông qua RESTful API với các điều kiện tìm kiếm.

7. Backend tiếp nhận request và truy vấn Database.
8. Database tìm kiếm các bản ghi cảm biến phù hợp với loại cảm biến và khoảng thời gian được yêu cầu.
9. Database trả kết quả về cho Backend.
10. Backend trả danh sách dữ liệu về Frontend.
11. Frontend hiển thị lịch sử dữ liệu cảm biến dưới dạng bảng.

Các thông tin được hiển thị gồm:

- ID dữ liệu.
- Tên/loại cảm biến.
- Giá trị đo được.
- Đơn vị đo.
- Thời gian đo.

ID	Cảm biến	Giá trị	Đơn vị	Thời gian
101	Nhiệt độ	26,5	°C	14:00:05
102	Nhiệt độ	27	°C	14:05:05
103	Nhiệt độ	28,5	°C	14:10:05
104	Nhiệt độ	29	°C	14:15:05

API đề xuất: GET api/data-sensor?type=temperature

c) Giao diện người dùng

MAI ANH DUC

MAIN MENU

Dashboard

Sensors

Control History

Profile

Sensors

Monitor sensor readings and environmental conditions.

Total 504 records

Filter

Sensor Type

All Sensors

Time Precision

Second

From

To

Apply Filter

Reset

Sensor	Type	Value	Unit	Time
Light Sensor	Light	0	lux	12/08/2026 00:00:02
Temperature Sensor	Temperature	26.7	°C	12/08/2026 00:00:06
Humidity Sensor	Humidity	67.5	%	12/08/2026 00:00:44
Temperature Sensor	Temperature	27.8	°C	12/08/2026 01:00:12
Light Sensor	Light	0	lux	12/08/2026 01:00:36
Humidity Sensor	Humidity	68.7	%	12/08/2026 01:00:51
Humidity Sensor	Humidity	69.3	%	12/08/2026 02:00:00
Temperature Sensor	Temperature	27.6	°C	12/08/2026 02:00:23
Light Sensor	Light	0	lux	12/08/2026 02:00:41
Humidity Sensor	Humidity	69.2	%	12/08/2026 03:00:08
Light Sensor	Light	0	lux	12/08/2026 03:00:22
Temperature Sensor	Temperature	27.4	°C	12/08/2026 03:00:45
Temperature Sensor	Temperature	27.6	°C	12/08/2026 04:00:02
Humidity Sensor	Humidity	71.2	%	12/08/2026 04:00:19
Light Sensor	Light	0.5	lux	12/08/2026 04:00:57

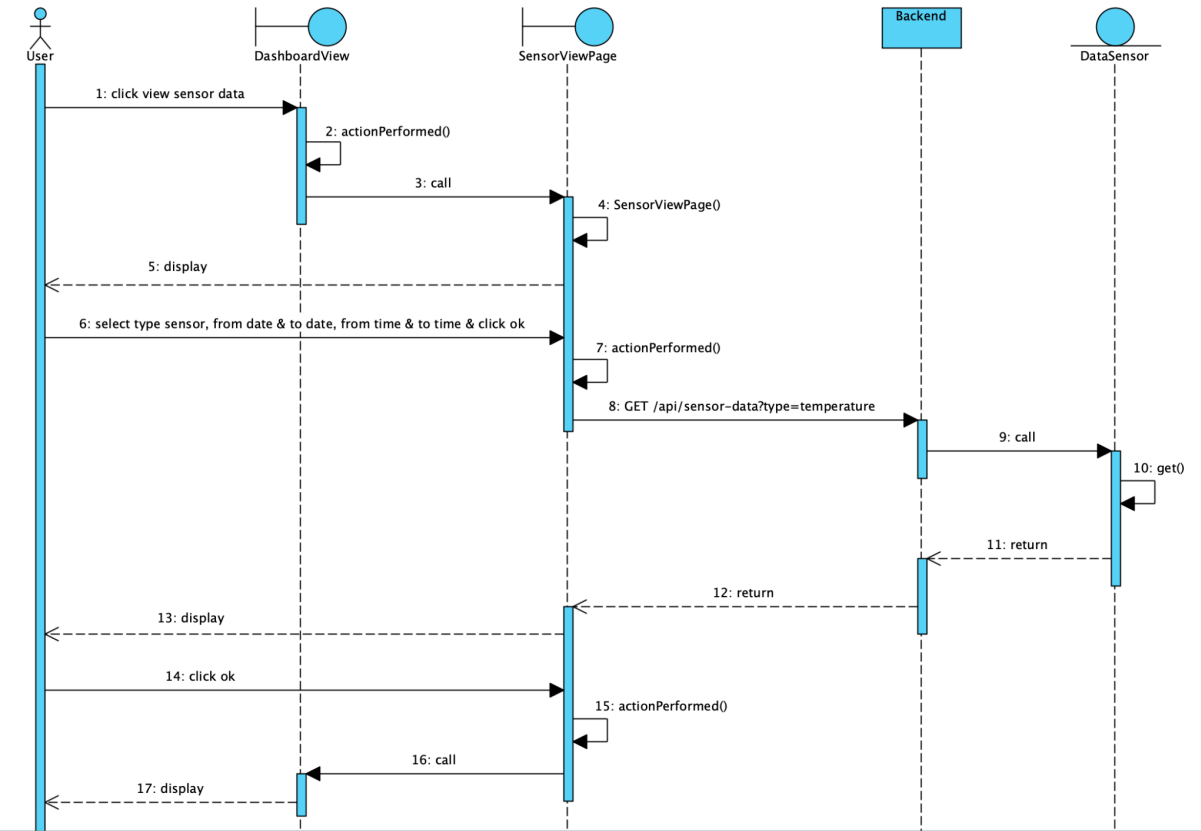
1-15 of 504 records < 1 2 3 4 5 ... 34 > 15 / page

Settings

Logout

Admin Super Admin

d) Sequence diagram



Mô tả tuần tự

1. Người dùng nhấn “View Sensor Data” trên DashboardView để truy cập chức năng xem dữ liệu cảm biến.
2. DashboardView tiếp nhận thao tác của người dùng và thực hiện xử lý sự kiện.
3. DashboardView gửi yêu cầu mở SensorViewPage.
4. SensorViewPage được khởi tạo để cung cấp giao diện xem dữ liệu cảm biến.
5. SensorViewPage hiển thị giao diện xem dữ liệu cảm biến cho người dùng.
6. Người dùng chọn loại cảm biến, ngày bắt đầu, ngày kết thúc, thời gian bắt đầu, thời gian kết thúc và nhấn OK.
7. SensorViewPage tiếp nhận thao tác và xử lý yêu cầu tìm kiếm.
8. SensorViewPage gửi request GET /api/sensor-data đến Backend với các điều kiện tìm kiếm.
9. Backend tiếp nhận request và gọi DataSensor để lấy dữ liệu phù hợp.
10. DataSensor truy vấn dữ liệu cảm biến theo loại cảm biến và khoảng thời gian được yêu cầu
11. DataSensor trả kết quả dữ liệu cảm biến về Backend.
12. Backend trả danh sách dữ liệu cảm biến về ViewHistorySensor.
13. ViewHistorySensor hiển thị lịch sử dữ liệu cảm biến cho người dùng.
14. Người dùng nhấn OK sau khi hoàn tất việc xem lịch sử cảm biến.
15. ViewHistorySensor tiếp nhận thao tác và xử lý sự kiện.
16. ViewHistorySensor gửi yêu cầu quay lại SensorViewPage.
17. SensorViewPage hiển thị lại giao diện cho người dùng.

III. Điều khiển thiết bị

1. **Bật tắt thiết bị**

a. Mô tả chức năng

Cho phép người dùng bật hoặc tắt thiết bị từ giao diện Web. Khi người dùng thực hiện thao tác, hệ thống gửi lệnh điều khiển đến ESP32 thông qua MQTT Broker để thiết bị thực hiện tương ứng.

b. Mô tả chi tiết

Luồng xử lý:

1. Người dùng chọn thiết bị cần điều khiển trên giao diện Dashboard.
2. Người dùng nhấn nút **ON/OFF** tương ứng với thiết bị.
3. Frontend gửi yêu cầu điều khiển đến Backend thông qua **RESTful API**.
4. Backend tiếp nhận và kiểm tra yêu cầu, bao gồm thiết bị có tồn tại và trạng thái hiện tại của thiết bị.
5. Backend publish lệnh điều khiển lên **MQTT Broker** thông qua topic điều khiển tương ứng với thiết bị.
6. ESP32 subscribe topic điều khiển và nhận lệnh từ MQTT Broker.
7. ESP32 xử lý lệnh và thực hiện bật hoặc tắt thiết bị.
8. Sau khi thực hiện, ESP32 gửi **trạng thái thực tế của thiết bị** về Backend thông qua MQTT.
9. Backend tiếp nhận trạng thái từ ESP32 và cập nhật trạng thái thiết bị trong hệ thống.
10. Backend lưu thông tin lần điều khiển vào **lịch sử hoạt động của thiết bị**.
11. Backend trả kết quả cho Frontend để cập nhật trạng thái thiết bị trên giao diện Dashboard.

Các thông tin được hiển thị gồm:

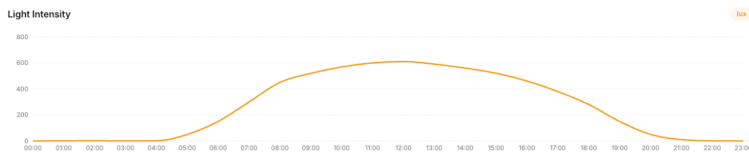
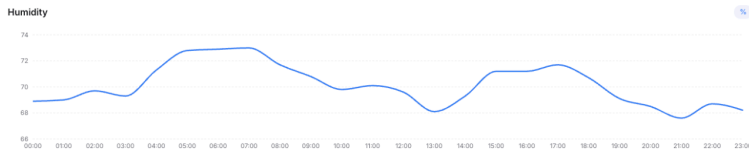
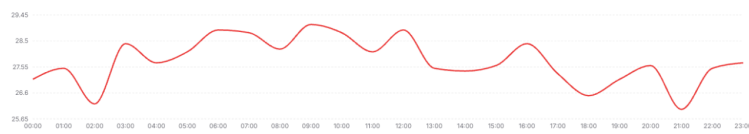
- Tên thiết bị: Tên của thiết bị được điều khiển.
- Trạng thái hiện tại: ON hoặc OFF.
- Hành động: TURN_ON hoặc TURN_OFF.
- Thời gian thực hiện: Ngày, giờ, phút, giây theo định dạng dd/MM/yyyy HH:mm:ss.
- Kết quả điều khiển: Thành công hoặc thất bại.
- Thông báo: Thông báo kết quả thực hiện lệnh cho người dùng.

API đề xuất: POST api/devices/{id}

c) Giao diện người dùng

Dashboard

Monitor your IoT system in real time.



Sensor Control

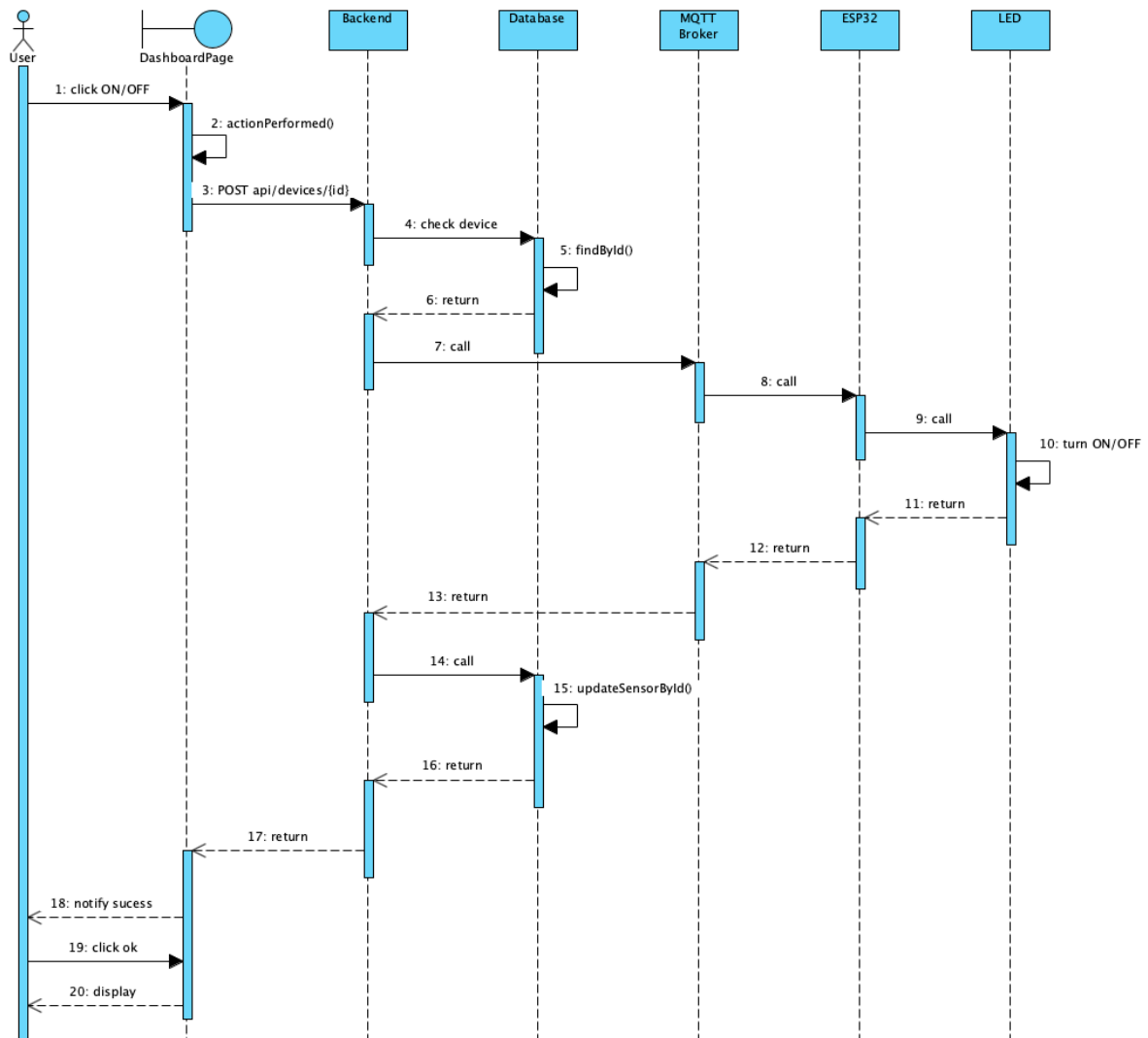
2 of 3 active

 **Temperature Sensor**
Temperature Sensor
Main Hall - °C
Active

 **Humidity Sensor**
Humidity Sensor
Main Hall - %
Active

 **Light Sensor**
Light Sensor
Main Hall - lux
Inactive

d) Sequence diagram



Mô tả tuần tự:

1. Bước 1: click ON/OFF Người dùng (User) thao tác nhấn nút ON hoặc OFF trên giao diện của thiết bị tại trang tổng quan (DashboardPage).
2. Bước 2: actionPerformed() Giao diện ghi nhận và xử lý sự kiện khi nút bấm được kích hoạt.
3. Bước 3: POST api/devices/{id} DashboardPage gọi API phương thức POST chứa ID của thiết bị gửi tới Backend để yêu cầu bật/tắt thiết bị.
4. Bước 4: check device Backend gửi yêu cầu đến Cơ sở dữ liệu (Database) để kiểm tra thông tin và trạng thái hiện tại của thiết bị.

5. Bước 5: findById() Database thực hiện truy vấn tìm kiếm thiết bị dựa trên ID được cung cấp.
6. Bước 6: return Database trả về kết quả tìm kiếm (thông tin thiết bị) cho Backend.
7. Bước 7: call Sau khi xác thực thiết bị hợp lệ, Backend publish lệnh điều khiển (call) tới MQTT Broker thông qua topic điều khiển tương ứng.
8. Bước 8: call MQTT Broker chuyển tiếp (gọi) lệnh điều khiển xuống vi điều khiển ESP32 đang subscribe topic đó.
9. Bước 9: call ESP32 nhận lệnh và xuất tín hiệu điều khiển trực tiếp đến thiết bị đầu cuối (LED).
10. Bước 10: turn ON/OFF Đèn LED thực hiện thay đổi trạng thái (bật lên hoặc tắt đi) theo tín hiệu điều khiển.
11. Bước 11: return Sau khi thực hiện, trạng thái thực tế của LED được trả về cho ESP32.
12. Bước 12: return ESP32 publish bản tin phản hồi chứa trạng thái mới của thiết bị lên MQTT Broker.
13. Bước 13: return MQTT Broker chuyển tiếp thông điệp phản hồi này về lại cho Backend.
14. Bước 14: call Backend nhận được trạng thái mới và gửi yêu cầu cập nhật dữ liệu vào Database.
15. Bước 15: updateSensorById() Database thực thi hàm cập nhật trạng thái mới nhất cho thiết bị (sensor/thiết bị) theo ID.
16. Bước 16: return Database trả về kết quả cập nhật thành công cho Backend.
17. Bước 17: return Backend phản hồi kết quả của toàn bộ luồng điều khiển (thành công) lại cho Frontend (DashboardPage).
18. Bước 18: notify success Giao diện DashboardPage hiển thị thông báo điều khiển thành công cho Người dùng.
19. Bước 19: click ok Người dùng tương tác nhấn "OK" để đóng thông báo.
20. Bước 20: display DashboardPage cập nhật lại giao diện và hiển thị trạng thái mới nhất của thiết bị (ON hoặc OFF) trên màn hình.

2. Xem lịch sử điều khiển thiết bị

a) Mô tả tính năng

Cho phép người dùng xem lại lịch sử các thao tác bật/tắt thiết bị trong hệ thống. Thông tin lịch sử bao gồm **tên thiết bị, hành động, trạng thái và thời gian thực hiện**, giúp người dùng theo dõi và kiểm tra các hoạt động điều khiển thiết bị đã diễn ra.

b) Mô tả chi tiết

Luồng xử lý:

1. Người dùng chọn chức năng Xem lịch sử điều khiển trên giao diện Lịch sử điều khiển.
2. Frontend gửi yêu cầu lấy lịch sử điều khiển đến Backend thông qua RESTful API.
3. Backend tiếp nhận và xử lý yêu cầu.
4. Backend truy vấn Database để lấy danh sách lịch sử điều khiển của các thiết bị.
5. Database thực hiện truy vấn bảng ControlHistory và trả về các bản ghi lịch sử.
6. Backend tiếp nhận và xử lý dữ liệu lịch sử từ Database.
7. Backend trả danh sách lịch sử điều khiển về Frontend.
8. Frontend nhận dữ liệu và hiển thị lịch sử điều khiển trên giao diện cho người dùng.
9. Người dùng có thể xem, theo dõi các lần bật/tắt thiết bị đã được thực hiện.

Các thông tin được hiển thị gồm:

- Tên thiết bị: Tên của thiết bị đã được điều khiển.
- Trạng thái: Trạng thái của thiết bị sau khi thực hiện lệnh, ON hoặc OFF.
- Hành động: Hành động đã thực hiện, TURN_ON hoặc TURN_OFF.
- Thời gian thực hiện: Ngày, giờ, phút, giây theo định dạng dd/MM/yyyy HH:mm:ss.
- Kết quả điều khiển: Kết quả của thao tác, SUCCESS hoặc FAILED.

- Thông báo: Thông tin mô tả kết quả hoặc trạng thái thực hiện thao tác.

API đề xuất: GET api/devices/control-history

c) Giao diện người dùng

MAI ANH DUC

MAIN MENU

Dashboard

Sensors

Control History

Profile

Settings

Logout

Device Control History

Track all device control activities.

Total 25 records

Filter

Device

All Devices

Action

All Actions

Status

All Status

Time Precision

Minute

From

To

From (DD/MM/YYYY HH:mm)

To (DD/MM/YYYY HH:mm)

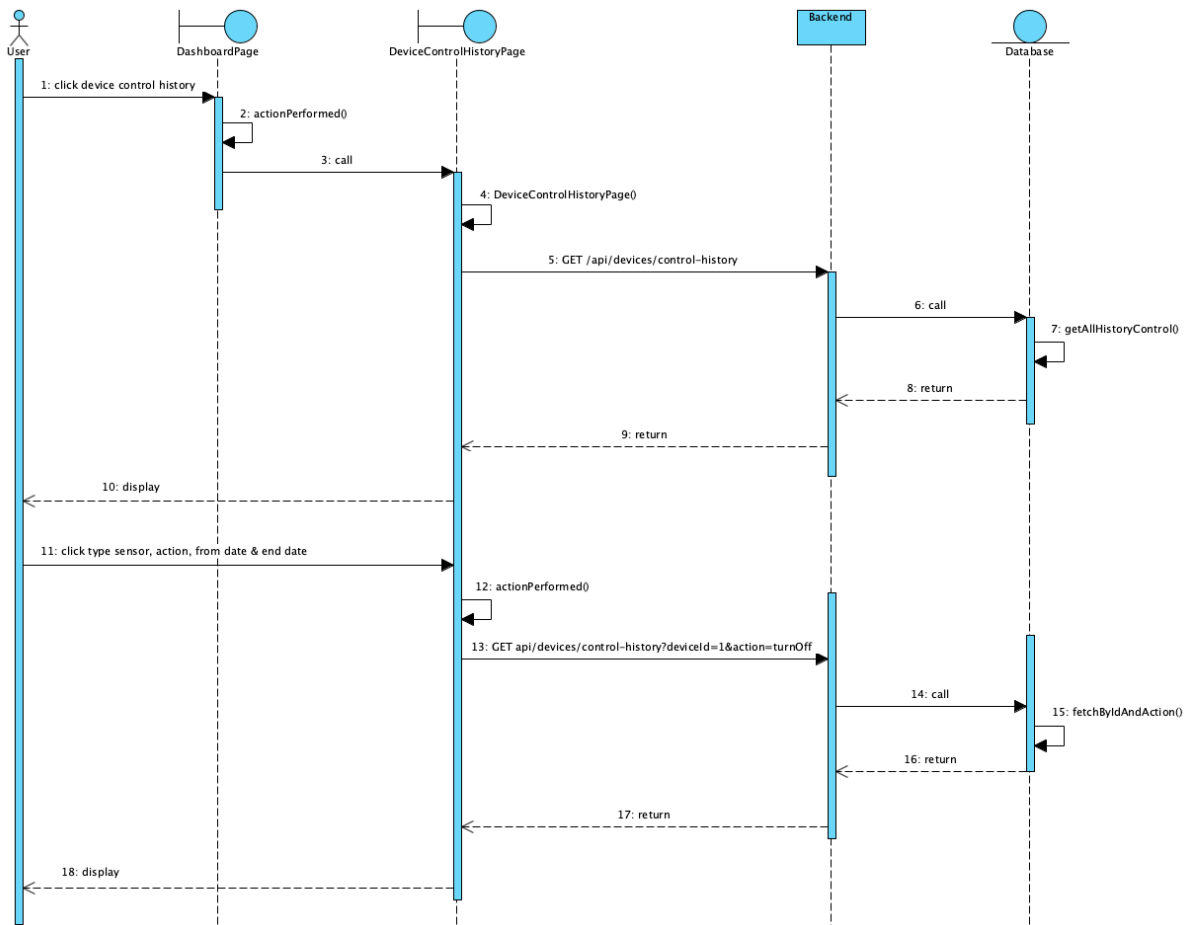
Apply Filter

Reset

User	Device	Action	Status	Time
admin	Temperature Sensor	TURN ON	SUCCESS	18/08/2026 18:20:30
admin	Temperature Sensor	TURN OFF	SUCCESS	18/08/2026 18:25:10
admin	Humidity Sensor	TURN ON	FAILED	18/08/2026 18:30:42
user01	Light Sensor	TURN ON	SUCCESS	18/08/2026 18:35:15
user01	Light Sensor	TURN OFF	SUCCESS	18/08/2026 18:40:00
admin	Humidity Sensor	TURN ON	SUCCESS	18/08/2026 19:00:00
admin	Humidity Sensor	TURN OFF	SUCCESS	18/08/2026 19:30:00
user02	Temperature Sensor	TURN ON	SUCCESS	17/08/2026 08:10:05
user02	Temperature Sensor	TURN OFF	FAILED	17/08/2026 08:15:22
admin	Light Sensor	TURN ON	SUCCESS	17/08/2026 09:00:00

1-10 of 25 records < 1 2 3 > 10 / page

d) Sequence diagram



Mô tả tuần tự:

1. Người dùng thao tác nhấn chọn mục xem lịch sử điều khiển thiết bị từ giao diện trang tổng quan.
2. Hệ thống ghi nhận sự kiện tương tác của người dùng.
3. Giao diện tổng quan thực hiện lệnh chuyển hướng sang trang chi tiết lịch sử điều khiển.
4. Giao diện trang lịch sử điều khiển thiết bị bắt đầu được khởi tạo.
5. Ngay trong quá trình khởi tạo, giao diện này tự động gửi một yêu cầu (HTTP GET) đến máy chủ (Backend) để lấy toàn bộ dữ liệu lịch sử.
6. Backend tiếp nhận yêu cầu và tiến hành gọi xuống Cơ sở dữ liệu.
7. Cơ sở dữ liệu thực thi truy vấn để quét và thu thập toàn bộ các bản ghi về lịch sử điều khiển.
8. Cơ sở dữ liệu trả danh sách dữ liệu vừa tìm được lại cho Backend.

9. Backend tiếp nhận, xử lý dữ liệu và gửi phản hồi kết quả về lại cho giao diện người dùng.
10. Giao diện hiển thị danh sách toàn bộ lịch sử điều khiển lên màn hình để người dùng theo dõi.
11. Người dùng thao tác trên bộ lọc của giao diện, chọn các tiêu chí mong muốn như: loại thiết bị, hành động (bật/tắt), mốc thời gian bắt đầu và kết thúc.
12. Giao diện ghi nhận sự thay đổi từ bộ lọc mà người dùng vừa thiết lập.
13. Dựa trên các tiêu chí đó, giao diện tự động tạo một yêu cầu mới, đính kèm các điều kiện lọc này vào đường dẫn API và gửi đến Backend.
14. Backend nhận yêu cầu và truyền chính xác các điều kiện lọc này xuống Cơ sở dữ liệu.
15. Cơ sở dữ liệu thực thi một truy vấn có điều kiện để quét và chỉ lấy ra những bản ghi khớp với yêu cầu.
16. Cơ sở dữ liệu trả danh sách dữ liệu đã được lọc lặp lại cho Backend.
17. Backend hoàn tất quá trình xử lý và trả gói dữ liệu lịch sử này về cho giao diện.
18. Giao diện cập nhật lại bảng dữ liệu trên màn hình, lúc này chỉ hiển thị những bản ghi thỏa mãn chính xác các tiêu chí tìm kiếm của người dùng.

CHƯƠNG 3. THIẾT KẾ CƠ SỞ DỮ LIỆU

I. Phân tích lớp thực thể

1. Phân tích các lớp thực thể

Hệ thống IoT gồm 5 lớp thực thể chính: User, Device, Sensor, SensorData và Action. Các lớp thực thể được xây dựng nhằm quản lý người dùng, thiết bị, cảm biến, dữ liệu cảm biến và lịch sử thao tác điều khiển trong hệ thống.

1.1 Lớp User

Lớp **User** đại diện cho người dùng sử dụng hệ thống IoT. Lớp này chịu trách nhiệm lưu trữ thông tin tài khoản, thông tin cá nhân và vai trò của người dùng trong hệ thống.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của người dùng
name	String	Họ và tên người dùng
email	String	Địa chỉ email
password	String	Mật khẩu đăng nhập
username	String	Tên đăng nhập
phone	String	Số điện thoại
avatar	String	Đường dẫn ảnh đại diện
github	String	Liên kết tài khoản GitHub
figma	String	Liên kết tài khoản Figma

role	String	Vai trò của người dùng, ví dụ ADMIN, USER
------	--------	---

Lớp User được sử dụng trong các chức năng đăng nhập, quản lý thông tin cá nhân và phân quyền người dùng.

1.2. Lớp Device

Lớp Device đại diện cho các thiết bị IoT được quản lý trong hệ thống. Các thiết bị có khả năng nhận lệnh điều khiển từ hệ thống, chẳng hạn như bật hoặc tắt đèn LED.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của thiết bị
name	String	Tên thiết bị
type	String	Loại thiết bị
status	String	Trạng thái hiện tại của thiết bị
mqttTopic	String	MQTT Topic dùng để giao tiếp với thiết bị
createdAt	DateTime	Thời gian tạo thiết bị

updatedAt	DateTime	Thời gian cập nhật thiết bị
-----------	----------	-----------------------------

Lớp Device phục vụ các chức năng quản lý và điều khiển thiết bị trong hệ thống.

1.3. Lớp Sensor

Lớp Sensor đại diện cho các cảm biến được sử dụng để thu thập thông tin từ môi trường.

Ví dụ:

- Cảm biến nhiệt độ.
- Cảm biến độ ẩm.
- Cảm biến ánh sáng.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh duy nhất của cảm biến
name	String	Tên cảm biến
type	String	Loại cảm biến
unit	String	Đơn vị đo
status	String	Trạng thái hoạt động của cảm biến
mqttTopic	String	MQTT Topic dùng để nhận dữ liệu

createdAt	DateTime	Thời gian tạo cảm biến
updatedAt	DateTime	Thời gian cập nhật cảm biến

Lớp Sensor chỉ lưu thông tin mô tả và trạng thái của cảm biến. Các giá trị đo thực tế được lưu trữ trong lớp SensorData.

1.4. Lớp SensorData

Lớp SensorData đại diện cho dữ liệu được thu thập từ cảm biến theo từng thời điểm. Đây là dữ liệu được sử dụng để theo dõi môi trường và hiển thị các biểu đồ trên Dashboard.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh của bản ghi dữ liệu
value	double	Giá trị đo được
time	DateTime	Thời điểm ghi nhận dữ liệu
sensorId	int	Mã của cảm biến tạo ra dữ liệu

Ví dụ, cảm biến nhiệt độ có thể tạo ra các dữ liệu:

- 10:00 → 26.1°C
- 10:01 → 26.4°C
- 10:02 → 26.7°C
- 10:03 → 27.0°C

Các dữ liệu này được sử dụng để xây dựng biểu đồ nhiệt độ theo thời gian trên Dashboard.

1.5. Lớp Action

Lớp Action đại diện cho một thao tác điều khiển thiết bị được thực hiện bởi người dùng. Lớp này được sử dụng để lưu lại lịch sử điều khiển thiết bị.

Thuộc tính	Kiểu dữ liệu	Mô tả
id	int	Mã định danh của thao tác
action	String	Hành động được thực hiện, ví dụ TURN_ON, TURN_OFF
status	String	Kết quả thực hiện, ví dụ SUCCESS, FAILED
time	DateTime	Thời điểm thực hiện thao tác
userId	int	Mã người dùng thực hiện thao tác
deviceId	int	Mã thiết bị được điều khiển

Ví dụ một bản ghi Action:

- User: admin
- Device: LED
- Action: TURN_ON
- Status: SUCCESS
- Time: 19/08/2026 10:30:15

Thông tin này được sử dụng trong chức năng Device Control History.

2. Phân tích mối quan hệ giữa các lớp thực thể

Sau khi xác định các lớp thực thể, hệ thống được xây dựng với các mối quan hệ giữa chúng như sau.

2.1. Quan hệ giữa User và Action

Một User có thể thực hiện nhiều thao tác điều khiển thiết bị. Tuy nhiên, mỗi Action chỉ được thực hiện bởi một User.

Do đó, quan hệ giữa User và Action là:

1-N (One-to-Many)

2.2. Quan hệ giữa Device và Action

Một Device có thể được điều khiển nhiều lần trong quá trình hoạt động của hệ thống. Mỗi Action chỉ tác động đến một Device.

Do đó, quan hệ giữa Device và Action là:

1-N (One-to-Many)

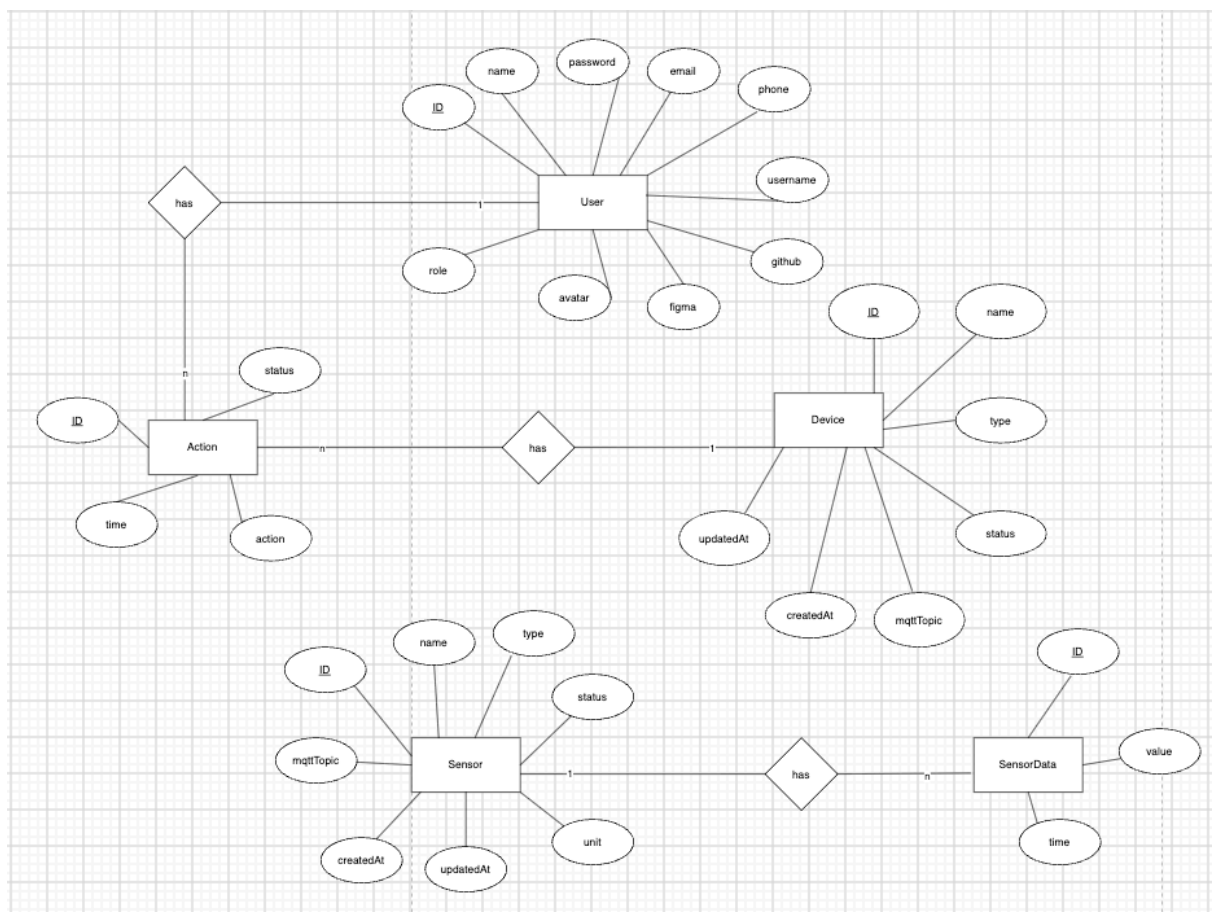
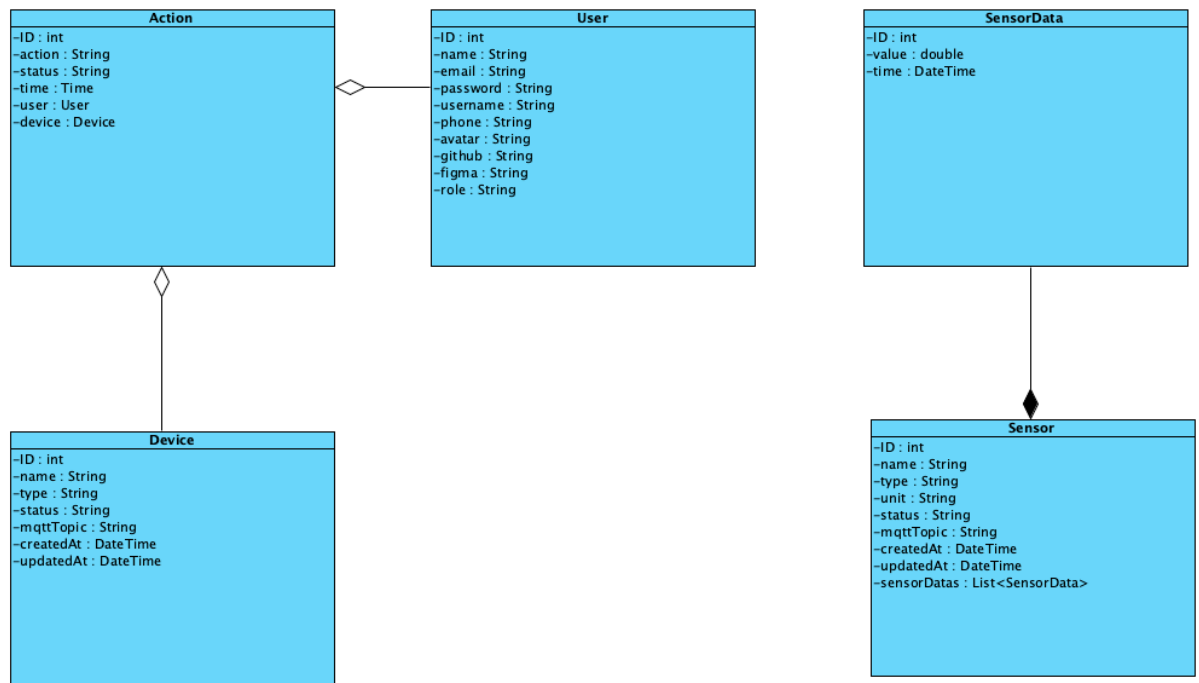
2.3. Quan hệ giữa Sensor và SensorData

Một Sensor có thể tạo ra nhiều bản ghi dữ liệu trong quá trình hoạt động. Ngược lại, mỗi SensorData chỉ thuộc về một Sensor.

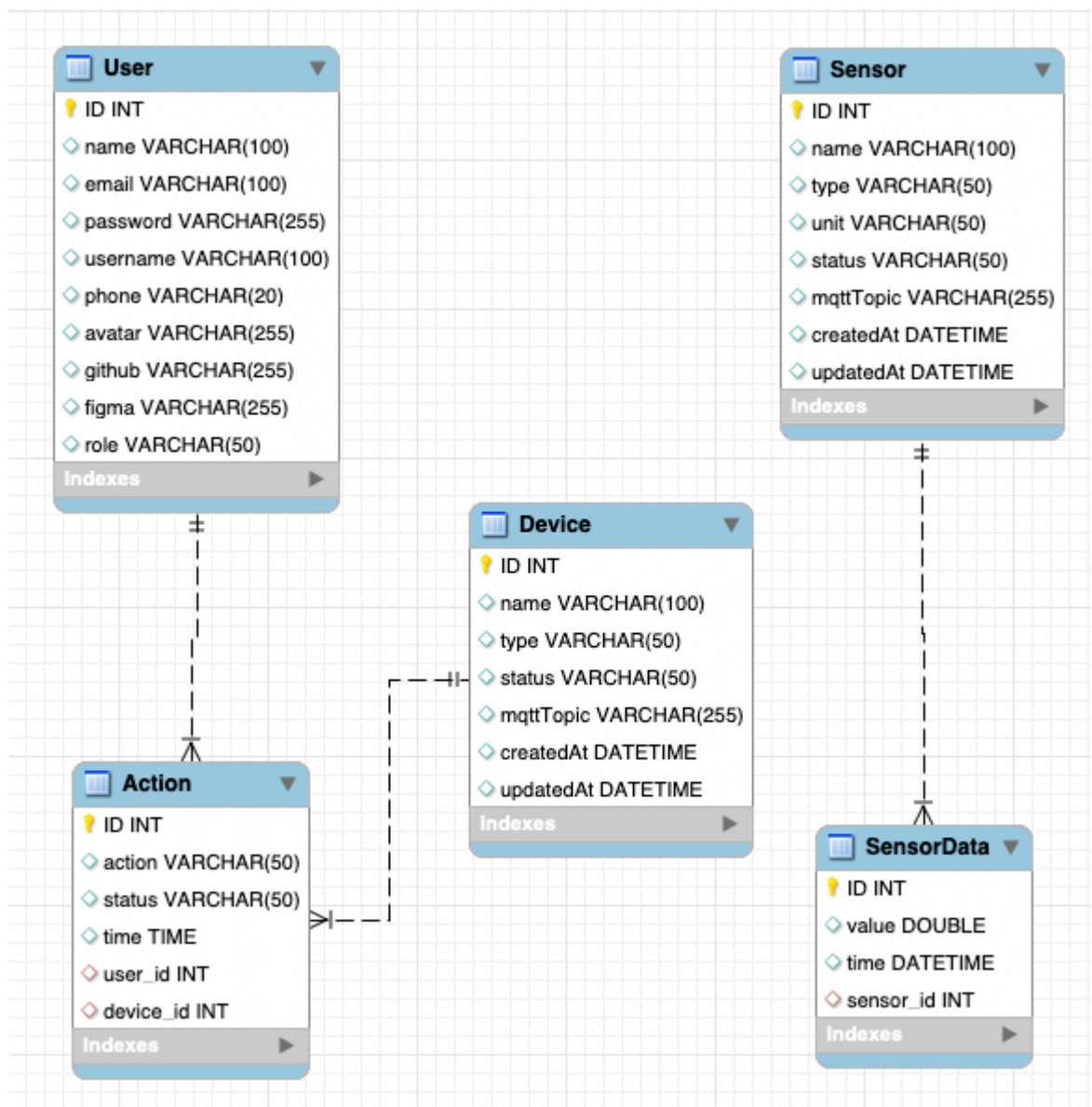
Do đó, quan hệ giữa Sensor và SensorData là:

1-N (One-to-Many)

II. Sơ đồ lớp thực thể



III. Sơ đồ ERD



CHƯƠNG 4. THIẾT KẾ API

1. API Đăng nhập

Thành phần	Nội dung
Method	POST
Endpoint	/api/auth/login
Mô tả	Xác thực người dùng bằng username/email và password, sau đó cấp JWT/Access Token nếu thông tin đăng nhập hợp lệ.
Authentication	Không yêu cầu
Authorization	Không yêu cầu

Request Body:

```
{  
  
  "username": "admin",  
  
  "password": "123456"  
}
```

Parameter	Kiểu dữ liệu	Bắt buộc	Vị trí	Mô tả
username	String	Có	Body	Username hoặc email dùng để đăng nhập
password	String	Có	Body	Mật khẩu

				của tài khoản
--	--	--	--	---------------

Response thành công

HTTP Status: 200 OK

```
{
  "message": "Login successfully",
  "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": 1,
    "username": "admin",
    "name": "Administrator",
    "role": "ADMIN"
  }
}
```

Field	Kiểu dữ liệu	Mô tả
message	String	Thông báo kết quả đăng nhập

accessToken	String	JWT/Access Token được cấp cho người dùng
user.id	int	ID người dùng
user.username	String	Username của người dùng
user.name	String	Tên người dùng
user.role	String	Vai trò của người dùng

Các mã phản hồi

HTTP Status	Ý nghĩa
200 OK	Đăng nhập thành công
400 Bad Request	Dữ liệu username/password không hợp lệ hoặc bị thiếu
401 Unauthorized	Username/email hoặc password không chính xác
500 Internal Server Error	Lỗi xảy ra phía Backend/Server

Response khi đăng nhập thất bại

HTTP Status: 401 Unauthorized

```
{  
  
  "message": "Username or password is incorrect"  
  
}
```

2. Profile

Thành phần	Nội dung
Method	GET
Endpoint	/api/auth/profile
Mô tả	Lấy thông tin cá nhân của người dùng đang đăng nhập.
Authentication	Required
Authorization	Người dùng đã đăng nhập

Request: Không yêu cầu Request Body.

Header: Authorization: Bearer <accessToken>

Parameter	Kiểu dữ liệu	Bắt buộc	Vị trí	Mô tả
-----------	--------------	----------	--------	-------

Authorization	String	Có	Header	JWT/Access Token của người dùng
---------------	--------	----	--------	---------------------------------

Response thành công

HTTP Status: **200** OK

JSON

```
{  
  
  "id": 1,  
  
  "name": "Nguyen Van A",  
  
  "username": "nguyenvana",  
  
  "email": "a@gmail.com",  
  
  "phone": "0912345678",  
  
  "avatar": "/uploads/avatar/1.jpg",  
  
  "github": "https://github.com/nguyenvana",  
  
  "figma": "https://figma.com/@nguyenvana",  
  
  "role": "USER"  
}
```

Field	Kiểu dữ liệu	Mô tả
-------	--------------	-------

id	int	ID người dùng
name	String	Họ và tên
username	String	Tên đăng nhập
email	String	Email
phone	String	Số điện thoại
avatar	String	Đường dẫn ảnh đại diện
github	String	Đường dẫn GitHub
figma	String	Đường dẫn Figma
role	String	Vai trò của người dùng

Các mã phản hồi

HTTP Status	Ý nghĩa
200 OK	Lấy thông tin thành công
401 Unauthorized	Token không tồn tại hoặc không hợp lệ
404 Not Found	Không tìm thấy người dùng
500 Internal Server Error	Lỗi phía Server

3. Xem dữ liệu cảm biến

Thuộc tính	Mô tả
Endpoint	/api/sensor-data/latest
Method	GET
Mục đích	Lấy các giá trị cảm biến mới nhất để hiển thị trên Dashboard.
Authentication	Có – yêu cầu người dùng đã đăng nhập và gửi JWT Token.

Content-Type	application/json
Request Body	Không có
Query Parameters	Không có
Path Parameters	Không có

Response thành công – 200 OK:

```
{
  "temperature": 28,
  "humidity": 72,
  "light": 430,
  "timestamp": "2026-08-19T14:20:05"
}
```

Trường	Kiểu dữ liệu	Mô tả
temperature	Double	Nhiệt độ mới nhất, đơn vị °C
humidity	Double	Độ ẩm mới nhất, đơn vị %

light	Double	Cường độ ánh sáng mới nhất, đơn vị lux
timestamp	DateTime	Thời gian cập nhật dữ liệu

Các mã trạng thái trả về:

Mã	Tên	Mô tả
200	OK	Lấy dữ liệu cảm biến thành công.
401	Unauthorized	Người dùng chưa đăng nhập hoặc JWT Token không hợp lệ/hết hạn.
403	Forbidden	Người dùng đã xác thực nhưng không có quyền truy cập API.
404	Not Found	Không tìm thấy dữ liệu cảm biến.
500	Internal Server Error	Có lỗi xảy ra phía Backend/Server.

4. Xem biểu đồ

Thuộc tính	Mô tả
Endpoint	/api/sensor-data
Method	GET
Mục đích	Lấy dữ liệu cảm biến theo loại cảm biến và khoảng thời gian được lựa chọn.
Authentication	Có – yêu cầu JWT Token.
Request Body	Không có
Query Parameters	type, from, to

Request Header:

Authorization: Bearer <JWT_TOKEN>

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả	Ví dụ
type	String	Có	Loại cảm biến cần xem	temperature

from	DateTime	Có	Thời điểm bắt đầu	2026-08-14T10:00:00
to	DateTime	Có	Thời điểm kết thúc	2026-08-14T14:20:00

Response thành công – 200 OK:

```
[
  {
    "type": "temperature",
    "value": 27.5,
    "timestamp": "2026-08-14T10:00:00"
  },
  {
    "type": "temperature",
    "value": 28.1,
    "timestamp": "2026-08-14T11:00:00"
  },
  {
    "type": "temperature",
    "value": 29.0,
    "timestamp": "2026-08-14T12:00:00"
  }
]
```

}

]

Các trường dữ liệu trả về

Trường	Kiểu dữ liệu	Mô tả
type	String	Loại cảm biến
value	Double	Giá trị đo được
timestamp	DateTime	Thời gian đo

Các mã trạng thái trả về

Mã	Tên	Mô tả
200	OK	Truy vấn dữ liệu thành công.
400	Bad Request	Tham số truyền vào không hợp lệ.
401	Unauthorized	Chưa xác thực hoặc JWT Token không hợp lệ/hết hạn.
403	Forbidden	Không có quyền truy cập API.

404	Not Found	Không tìm thấy dữ liệu phù hợp.
500	Internal Server Error	Lỗi xảy ra phía Backend/Server.

5. Xem lịch sử dữ liệu cảm biến

Thuộc tính	Mô tả
Method	GET
Endpoint	/api/sensor-data/history
Mô tả	Lấy danh sách dữ liệu cảm biến đã được lưu trong Database theo loại cảm biến và khoảng thời gian.
Authentication	Có – yêu cầu JWT Token
Request Body	Không có
Query Parameters	type, from, to, page, size

Query Parameters

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả

type	String	Không	Loại cảm biến (temperature, humidity, light)
from	DateTime	Không	Thời gian bắt đầu
to	DateTime	Không	Thời gian kết thúc
page	Integer	Không	Số trang cần lấy
size	Integer	Không	Số bản ghi trên một trang

Ví dụ Request

HTTP

GET

/api/sensor-data/history?type=temperature&from=2026-08-14T10:00:00&to=2026-08-14T14:20:00&page=0&size=20

Response – 200 OK

JSON

{

"content": [

{

"id": 101,

"type": "temperature",

"value": 26.5,

```
"unit": "°C",

"timestamp": "2026-08-14T14:00:05"

},

{

  "id": 102,

  "type": "temperature",

  "value": 27.0,

  "unit": "°C",

  "timestamp": "2026-08-14T14:05:05"

}

],

"page": 0,

"size": 20,

"totalElements": 100,

"totalPages": 5

}
```

Các mã trả về

Mã	Ý nghĩa

200 OK	Lấy dữ liệu thành công
400 Bad Request	Tham số truyền vào không hợp lệ
401 Unauthorized	Chưa xác thực hoặc JWT không hợp lệ
403 Forbidden	Không có quyền truy cập
404 Not Found	Không tìm thấy dữ liệu
500 Internal Server Error	Lỗi phía Server

6. Điều khiển thiết bị

Thuộc tính	Mô tả
Method	POST
Endpoint	/api/devices/{id}/control
Mô tả	Gửi lệnh bật hoặc tắt thiết bị thông qua Backend và MQTT Broker.

Authentication	Có – yêu cầu JWT Token
Path Parameters	id – ID của thiết bị cần điều khiển
Request Body	Có
Content-Type	application/json

Request Body

JSON

```
{
  "action": "TURN_ON"
}
```

Trong đó action nhận một trong hai giá trị:

- TURN_ON – bật thiết bị.
- TURN_OFF – tắt thiết bị.

Ví dụ Request

POST /api/devices/1/control

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

```
{
```

```
"action": "TURN_ON"  
}
```

Response – 200 OK

JSON

```
{  
  
"deviceId": 1,  
  
"deviceName": "Đèn phòng khách",  
  
"action": "TURN_ON",  
  
"status": "ON",  
  
"result": "SUCCESS",  
  
"timestamp": "2026-08-19T14:30:05",  
  
"message": "Device turned on successfully"  
}
```

Các mã trả về

Mã	Ý nghĩa
200 OK	Gửi và thực hiện lệnh điều khiển thành công
400 Bad Request	Action không hợp lệ hoặc dữ liệu request không hợp lệ

401 Unauthorized	Chưa xác thực hoặc JWT không hợp lệ
403 Forbidden	Người dùng không có quyền điều khiển thiết bị
404 Not Found	Không tìm thấy thiết bị
409 Conflict	Trạng thái thiết bị không phù hợp để thực hiện lệnh
500 Internal Server Error	Lỗi phía Backend/MQTT

7. Xem lịch sử điều khiển thiết bị

Thuộc tính	Mô tả
Method	GET
Endpoint	/api/devices/control-history
Mô tả	Lấy danh sách lịch sử các thao tác điều khiển thiết bị đã được lưu trong Database.
Authentication	Có – yêu cầu JWT Token

Request Body	Không có
Query Parameters	deviceId, from, to, page, size

Query Parameters

Tham số	Kiểu dữ liệu	Bắt buộc	Mô tả
deviceId	Long	Không	ID thiết bị cần xem lịch sử
from	DateTime	Không	Thời gian bắt đầu
to	DateTime	Không	Thời gian kết thúc
page	Integer	Không	Số trang
size	Integer	Không	Số bản ghi trên một trang

Ví dụ Request

GET

/api/devices/control-history?deviceId=1&from=2026-08-14T00:00:00&to=2026-08-19T23:59:59&page=0&size=20

Authorization: Bearer <JWT_TOKEN>

Response – 200 OK

JSON

```
{  
  
  "content": [  
  
    {  
  
      "id": 101,  
  
      "deviceId": 1,  
  
      "deviceName": "Đèn phòng khách",  
  
      "action": "TURN_ON",  
  
      "status": "ON",  
  
      "result": "SUCCESS",  
  
      "timestamp": "2026-08-19T14:30:05",  
  
      "message": "Device turned on successfully"  
    },  
  
    {  
  
      "id": 102,  
  
      "deviceId": 1,  
  
      "deviceName": "Đèn phòng khách",  
  
      "action": "TURN_OFF",  
  
      "status": "OFF",  
  
      "result": "SUCCESS",  
  
      "timestamp": "2026-08-19T15:10:20",  
    }  
  ]  
}
```

```
    "message": "Device turned off successfully"

  },

  "page": 0,

  "size": 20,

  "totalElements": 2,

  "totalPages": 1

}
```

Các mã trả về

Mã	Ý nghĩa
200 OK	Lấy lịch sử điều khiển thành công
400 Bad Request	Tham số tìm kiếm không hợp lệ
401 Unauthorized	Chưa xác thực hoặc JWT không hợp lệ
403 Forbidden	Không có quyền xem lịch sử
404 Not Found	Không tìm thấy dữ liệu lịch sử
500 Internal Server Error	Lỗi phía Backend/Server

